PERSONAL INFORMATION MANAGEMENT (PIM) SYSTEM

Software Requirements Specification

Document	Software Requirements Specification
Version	1.4
Date	8 November 2026
Group	Group [number]
Status	Final

Group members

Name	Student ID
[Member 1 name]	[Student ID]
[Member 2 name]	[Student ID]
[Member 3 name]	[Student ID]
[Member 4 name]	[Student ID]

Contents

1 Preface **3**
1.1 Readers . . . 3
1.2 Version history . . . 3
1.3 Conventions . . . 3

2 Introduction **4**
2.1 Purpose of the system . . . 4
2.2 Scope . . . 4
2.3 Operating environment . . . 4

3 Glossary **5**

4 User requirements definition **6**

5 System architecture **6**

6 System requirements specification **7**
6.1 Creating PIRs (US1–US5) . . . 7
6.2 Modifying PIRs (US6) . . . 9
6.3 Searching (US7) . . . 10
6.4 Printing (US8) . . . 11
6.5 Deleting (US9) . . . 11
6.6 Storing (US10) . . . 12
6.7 Loading (US11) . . . 12
6.8 User interface . . . 13
6.9 Alarm Alerts (derived from US4) . . . 14
6.10 Non-functional requirements . . . 14

1 Preface

1.1 Readers

This specification is written for three groups of readers:

- the course instructor and teaching assistants, who assess the system against Appendix B of the project description;
- the group members who design, implement, and test the system;
- anyone who later maintains or tests the system.

Readers should be familiar with basic software-engineering terms. No knowledge of Python is needed.

1.2 Version history

Version	Date	Changes
1.0 draft	23 Sep 2026	First complete draft, covering user stories US1–US11, the in-process Alarm Alert, and the non-functional constraints of the brief.
1.1 draft	24 Sep 2026	Paths may start with ~, and the status line names the absolute path (FR-29, FR-34). Added the accelerators `W` (save as) and `o` (load) (FR-38) and the folder browser (new FR-39); the former FR-39 – FR-42 became FR-40 – FR-43. Verification lines name the tests.
1.2 draft	24 Sep 2026	This SRS became the only product specification in the source tree. Added the glossary term Alarm Alert, the selection errors in FR-26, and the list of covered invalid inputs in NFR-4.
1.3 draft	25 Sep 2026	HKT falls back to a fixed UTC+8 offset when the computer has no time-zone database (glossary, NFR-6). Every requirement has a verification line.
1.4	8 Nov 2026	Release version, checked line by line against the implementation. A missing start or alarm time is now reported by name (FR-10). Corrected the path messages (FR-33), the end of redirected input (FR-37), the command aliases (FR-38), the differences between the two interfaces (FR-40, FR-41), and when a dismissed alert reappears (FR-43). Verification lines name the current tests.

1.3 Conventions

- **Requirement ids.** Functional requirements are numbered `FR-n` and non-functional requirements `NFR-n`.
- **"Shall".** Every requirement is stated with *shall* and can be checked by a test or by inspection. Its *Verification* line says how.
- **Rationale.** Italic text under a requirement explains why the requirement exists. It is not itself a requirement.
- **Literal text.** Commands, field names, messages, and other text that the user types or reads are set in `monospace`.

2 Introduction

2.1 Purpose of the system

The PIM is a **single-user, command-line personal information manager**. One person runs it in a terminal on one computer and uses it to:

- create four kinds of personal information record (PIR): notes, tasks, events, and contacts;
- modify, search, print, and delete those records;
- store the records in a file with the extension `.pim` and load them again later.

All the information the user cares about is therefore kept in one place, and it survives from one run of the program to the next (US1).

2.2 Scope

In scope. Everything that user stories US1–US11 of Appendix B require, plus one derived feature: while the program runs, it shows an **Alarm Alert** for every Event alarm that is overdue or due within the next 15 minutes. The alert is what makes the alarms of US4 useful in practice; it is not a new user story.

Out of scope. The brief gives no credit for additional features, so the system does not provide:

- a graphical user interface, or any third-party terminal library;
- networking, multiple users, or synchronisation;
- recurring events;
- links between PIRs;
- unique labels or titles, or fuzzy (edit-distance) search;
- changing the type of an existing PIR;
- operating-system notifications, background processes, or snoozing an alarm.

2.3 Operating environment

The system is a single Python 3 process that uses only the standard library. The user types commands in a terminal:

- on an interactive terminal (TTY), the system shows a full-screen text interface built with the standard `curses` module;
- when input or output is redirected, as in the automated tests, it uses a plain line-by-line interface with the same commands.

The system reads and writes `.pim` files on the local file system. It has no other external interfaces.

3 Glossary

Term	Meaning
PIM	The system: one Working Collection of PIRs that can be stored in, and loaded from, a PIM File.
PIR	Personal Information Record. Every PIR has exactly one type: Note, Task, Event, or Contact.
Id	The unique integer identity that the system assigns to a PIR. It is stored in the PIM File, stays the same across save and load, and is never reused after a delete.
Working Collection	The PIRs held by the running program, as opposed to any file on disk.
Note	A PIR whose only content is a required plain-text body, the field `text`. A Note has no separate title.
Task	A PIR with a required `description` and an optional `deadline`.
Event	A PIR with a required `description`, a required starting time `start`, and zero or more Alarms.
Contact	A PIR with a required `name` and an optional `address` and `mobile` number.
Name	The person name of a Contact. It need not be unique and is not an identity.
Alarm	A reminder attached to an Event. It is either *Relative* (at the start, or a given time before it) or *Absolute* (a fixed instant).
Alarm Alert	A notice shown while the program runs for an Alarm that is OVERDUE (its Effective Alarm Time has passed) or SOON (it is due within 15 minutes). It stays until the user dismisses it. Dismissals are not stored.
Effective Alarm Time	The instant at which an Alarm fires: `start` minus the duration for a Relative alarm, or the stored instant for an Absolute alarm.
Display Name	A short label used in lists and never stored: the first line of the body for a Note, the description for a Task or Event, and the name for a Contact.
Current Result	The list of PIRs on screen. It starts as the whole Working Collection; a search replaces it with the matching PIRs.
Selection	The one PIR, chosen by Id or by its row in the Current Result, that `modify`, `print`, and `delete` act on.
Search Criterion	A condition on the type and field values of a PIR, written on one line (FR-16).
Bound File	The `.pim` file most recently saved or loaded, if any.
Dirty	The state in which the Working Collection has changes that are not yet saved.
HKT	Hong Kong Time, the IANA time zone `Asia/Hong_Kong`: UTC+8 for present-day times, with no daylight saving since 1979. On a computer without a time-zone database, a fixed UTC+8 offset is used. HKT is the default time zone.
PIM File	A UTF-8 JSON file with the extension `.pim` that stores a Working Collection.
Status line	The one-line area in which the system reports the outcome of every command.

4 User requirements definition

The system shall support the following user stories, quoted from Appendix B of the project description. Section 6 refines each story into system requirements.

Id	User story	Refined by
US1	As a user, I want to create different types of personal information records (PIRs) in the PIM so that all the information I care about can be managed in a single location.	FR-1 – FR-3, FR-10, FR-11
US2	As a user, I want to create new plain texts as PIRs so that I can use the PIM to take quick notes.	FR-4
US3	As a user, I want to create new tasks with the corresponding descriptions and deadlines as PIRs so that I can use the PIM to manage my to-dos.	FR-5
US4	As a user, I want to create new events with the corresponding descriptions, starting times, and alarms as PIRs so that I can use the PIM to manage my schedule.	FR-6 – FR-8, FR-41 – FR-43
US5	As a user, I want to create new contacts with the corresponding names, addresses, and mobile numbers as PIRs so that I can use the PIM to manage my contacts.	FR-9
US6	As a user, I want to modify the data in existing PIRs so that I can keep the PIRs up to date.	FR-12 – FR-15
US7	As a user, I want to search for PIRs based on criteria concerning their types and the data stored in their fields. A criterion may check whether a piece of text (stored in a note, a description, a name, an address, or a mobile number) contains a string, whether a time (stored in a deadline, a starting time, or an alarm) is before (<), after (>), or equal to (=) another given point in time, or whether a condition combining multiple other conditions via logical connectors and (&&), or (||), and negation (!) is satisfied.	FR-16 – FR-23
US8	As a user, I want to print out detailed information about a specific PIR or all PIRs.	FR-24 – FR-26
US9	As a user, I want to delete a specified PIR.	FR-27, FR-28
US10	As a user, I want to store the PIRs in a file with the extension name ".pim" so that I can access them using the PIM in the future.	FR-29 – FR-33
US11	As a user, I want to load the PIRs from a file with the extension name ".pim" so that I can continue working with the PIRs I stored earlier.	FR-34 – FR-37

The user-interface requirements FR-38 – FR-40 serve every story. The non-functional requirements NFR-1 – NFR-10 (Section 6.10) apply to the whole system.

5 System architecture

The system follows the **Model–View–Controller** (MVC) pattern, with one Python package for each component. The design document describes the architecture in full; this section only names the parts so that the requirements can refer to them.

- `model` holds the data and every rule about it:
  - the Working Collection (class `PIM`) and the four PIR types;
  - alarms;
  - the Search Criterion parser and matcher;
  - reading and writing the PIM File.

  It has no user interface and imports neither `view` nor `controller`, so it can be unit-tested on its own.
- `controller` turns one completed user action into calls on the model. It keeps the Current Result and the Selection, and it reports the outcome of every action, success or failure, as one status-line message.
- `view` draws the terminal screen, reads keys and lines, runs the event loop, and shows Alarm Alerts. It sends every action to the controller. From the model it uses only value types: it displays PIRs and alerts, and it builds alarm values from the user's answers. It also reports input that never reaches the controller, such as an unknown command or an empty path.
- `pim.py` is the composition root. It creates `PIM()`, then `App(pim)`, then `Terminal(app)`, and runs the terminal.

6 System requirements specification

6.1 Creating PIRs (US1–US5)

FR-1 The system shall support exactly four PIR types: Note, Task, Event, and Contact. The command `create note|task|event|contact` shall start creating a PIR of the named type. `create` without a type shall first ask for the type, and an unknown type shall be rejected with `unknown PIR type: <type>`.

Verification unit tests `NoteTests`, `TaskTests`, `EventTests`, and `ContactTests` in `tests/unit/test_pir.py`; e2e test `test_create_type_prompt_and_search_criterion_prompt`.

FR-2 When a PIR is created, the system shall give it an Id equal to the next Id of the Working Collection and then increase the next Id by one. The first Id of a new collection shall be 1.

Verification unit tests `test_four_types_receive_monotonic_ids` and `test_failed_create_does_not_consume_id_or_dirty`.

FR-3 The system shall ask for the fields of a new PIR one prompt at a time. On an optional field, an empty answer shall leave the field unset.

Rationale: prompted entry needs no syntax to remember, and each field is named on the screen.

Verification integration test `test_create_note_wizard_updates_current_result`; e2e create scripts in `tests/e2e/test_user_flows.py`.

FR-4 **Note**

A Note shall have one required field, `text`. The system shall reject a Note whose text is empty or consists only of white space. The Display Name of a Note shall be the first line of its text.

Verification unit tests `NoteTests`.

FR-5 Task

A Task shall have a required `description` and an optional `deadline`.

Rationale for the optional deadline: many to-dos have no date, and forcing an invented one would make time searches such as `deadline < T` return false matches. When a deadline is given, US3 is fully met.

Verification unit tests `TaskTests`.

FR-6 Event

An Event shall have a required `description`, a required `start`, and zero or more Alarms. After each alarm, the system shall ask whether to add another.

Rationale for allowing no alarm: not every event needs a reminder. Any number of alarms, of either kind, can be added.

Verification unit tests `EventTests`.

FR-7 Alarm kinds

Each Alarm shall be one of the following:

- *Relative*: a whole number `amount` that is not negative, and a `unit` of `minute`, `hour`, `day`, or `week`. The alarm fires that long **before** `start`, and an amount of 0 means *at the start*. A negative amount (after the start) shall be rejected with `relative alarm cannot be after start`, and an amount that is not a whole number with `relative alarm amount must be an integer`.
- *Absolute*: a stored instant, independent of `start`.

Verification unit tests in `tests/unit/test_alarms.py` and `test_relative_after_start_rejected`.

FR-8 Alarm range

The system shall reject an Event if any Effective Alarm Time falls outside the representable range of dates (years 1 – 9999), for example 999999999 weeks before the start. The status message shall be `alarm time is out of range`.

Verification unit test `test_huge_relative_amount_fails_create_without_mutation`; e2e test `test_overflowing_relative_alarm_fails_create_and_session_continues`.

FR-9 Contact

A Contact shall have a required `name` and an optional `address` and `mobile`. Two Contacts may have the same name.

Verification unit tests `ContactTests`.

FR-10 Missing values

An empty answer, or one that consists only of white space, shall count as a missing value. Leading and trailing spaces shall be removed from stored text. If a required field is missing, the create shall fail with a message that names the field, for example `description is required` or `start is required`.

Verification unit test `test_validation_messages_name_the_field_or_value_at_fault`; unit tests for each PIR type.

FR-11 **Date and time input**

The system shall accept a date and time in ISO 8601 form or as `YYYY-MM-DD HH:MM`, with or without a time-zone offset:

- a value without an offset shall be read as Hong Kong Time;
- seconds shall be dropped, because the system works to the minute;
- a value that cannot be read shall be rejected with `invalid datetime: <value>`;
- a value that cannot be expressed in both HKT and UTC within years 1 – 9999, such as `9999-12-31T23:59-10:00` or `0001-01-01 00:10`, shall be rejected with `datetime out of range`, so that every saved value can be loaded again (FR-35).

Inside a Search Criterion, an unreadable or out-of-range value shall be reported as `search syntax error: invalid datetime: <value>`. On a TTY, date-time fields of a PIR shall also offer a calendar picker.

Verification unit tests `ParseDatetimeTests`.

6.2 Modifying PIRs (US6)

FR-12 The command `modify` shall ask for each field of the selected PIR in turn and show its current value. For each field:

- pressing Enter alone shall keep the current value;
- on an optional field, the answer `none` or an answer of only white space shall clear the field;
- on a required field, an answer of only white space shall fail with the message of FR-10;
- for an Event, the system shall ask whether to replace the alarm list. Answering yes and then adding no alarm removes every alarm.

Verification integration test `test_modify_keeps_id_and_empty_enter_keeps_text`; e2e modify scripts in `tests/e2e/test_user_flows.py`.

FR-13 Modifying a PIR shall never change its Id or its type. The `modify` command shall not offer the type as a field, and a request through the model interface to change the type shall fail with `PIR type cannot be changed`.

Rationale: the Id is the only identity (design document, Section 5.3). A record of another type is a different record, so the user deletes the PIR and creates a new one.

Verification unit tests `test_type_cannot_change` and `test_failed_modify_type_leaves_pir_unchanged`.

FR-14 When the `start` of an Event changes, the Effective Alarm Times of its Relative alarms shall move with it, and those of its Absolute alarms shall stay the same.

Verification unit test `test_relative_effective_times_move_with_start`.

FR-15 A modify that fails validation shall leave the PIR exactly as it was. A modify that changes nothing shall not mark the collection dirty.

Verification unit tests for modify in `tests/unit/test_pim.py`.

6.3 Searching (US7)

FR-16 Criterion grammar

The command `search <criterion>` shall accept one criterion line with the following grammar, in which `!` binds most tightly, then `&&`, then `||`:

```
criterion  := or
or         := and ( "||" and )*
and        := not ( "&&" not )*
not        := "!" not | primary
primary    := "(" criterion ")" | atom
atom       := "type" "=" ("note"|"task"|"event"|"contact")
            | textfield "contains" STRING
            | "contains" STRING
            | timefield ("<"|">"|"=") DATETIME
textfield  := "text" | "description" | "name" | "address" | "mobile"
timefield  := "deadline" | "start" | "alarm"
STRING     := '"' characters '"'
```

Keywords, field names, and type names shall be case-insensitive. Inside a STRING, a backslash shall make the next character literal, so `\"` stands for a quotation mark and `\\` for a backslash; `\n` stands for a line break. A DATETIME has one of the forms of FR-11 and may be written in quotation marks.

Verification parser unit tests in `tests/unit/test_search.py`.

FR-17 Type

`type = T` shall match exactly the PIRs of type T.

FR-18 Contains

`f contains "s"` shall match a PIR that has the text field `f` when `s`, after Unicode case folding, is a substring of the folded field value. The match shall be neither fuzzy nor dependent on the locale. A PIR without field `f` shall not match.

FR-19 Unqualified contains

`contains "s"` shall match a PIR when any of its text fields contains `s` in the sense of FR-18.

FR-20 Time comparison

`f < T`, `f > T`, and `f = T` shall compare instants, not clock readings, to the minute; `start = 2026-09-14 18:30` therefore matches the same Events as `start = 2026-09-14T10:30Z`. A comparison on a field that the PIR does not have, or has not set (such as a Task without a deadline), shall be false. For `alarm`, an Event shall match if **any** of its Effective Alarm Times satisfies the comparison.

FR-21 Logic

`A && B`, `A || B`, `!A`, and parentheses shall combine criteria with their usual Boolean meaning and the precedence of FR-16.

Verification of FR-17 – FR-21 unit tests in `tests/unit/test_search.py` on the shared six-PIR fixture (`tests/fixture.py`).

FR-22 **Result**

A successful search shall replace the Current Result with the matching PIRs in increasing Id order, select the first match, and report `<n> match(es)`. The command `clear` shall restore the whole Working Collection as the Current Result.

FR-23 **Syntax error**

A criterion that does not follow FR-16 shall be reported as `search syntax error: <reason>`, and the Current Result shall stay unchanged. In the full-screen interface, the typed criterion shall stay in the input field so that it can be corrected.

Verification of FR-22 – FR-23 controller unit tests in `tests/unit/test_app.py`; integration tests in `tests/integration/test_app.py`; curses unit test `test_search_syntax_error_keeps_the_typed_line`.

6.4 Printing (US8)

FR-24 The command `print` shall print every field of the selected PIR, with its Id and type. For an Event, it shall print each Alarm's kind and its Effective Alarm Time.

FR-25 The command `print all` shall print every field of every PIR in the **Current Result**, in list order. When no search is active (at start-up, after `clear`, or after `load`), the Current Result is the whole Working Collection, so `print all` prints all PIRs.

Rationale: US8 asks to print "all PIRs". Printing the Current Result also lets the user print all the PIRs that match a search, which is the more useful reading. Running `clear` first prints every PIR.

FR-26 `print`, `modify`, and `delete` with no Selection shall fail with `no PIR selected`. Selecting an Id that does not exist shall fail with `no PIR with Id <n>`, and a row number outside the Current Result with `no row <n> in Current Result`. In both cases the Selection shall stay unchanged.

Verification of FR-24 – FR-26 integration tests in `tests/integration/test_app.py`, including `test_select_row_is_not_identity`; e2e print scripts in `tests/e2e/test_user_flows.py`.

6.5 Deleting (US9)

FR-27 The command `delete` shall ask for confirmation (`y` or `n`) and then remove the selected PIR. Answering `n` shall leave the collection unchanged.

FR-28 An Id shall never be reused, not even after its PIR is deleted, or after a save and a load.

Verification of FR-27 – FR-28 unit tests `test_delete_does_not_reuse_id` and `test_next_id_survives_delete_then_save_load`; e2e test `test_delete_no_keeps_yes_removes_and_id_is_not_reused`.

6.6 Storing (US10)

FR-29 `save as <path>` shall write the Working Collection to `<path>`, adding `.pim` when the path does not already end in `.pim`, and the path shall become the Bound File. `save` shall write to the Bound File without asking, or behave as `save as` when there is none. A typed path may be absolute, start with ~ for the home folder, or be relative to the folder from which the system was started. Missing folders on the path shall be created. On a TTY, `save as` without a path shall open the folder browser (FR-39); the line interface shall ask for a typed path. After a successful save, the status line shall name the absolute path that was written.

FR-30 Whenever a save would replace an existing file that is not the Bound File, the system shall ask for confirmation first.

FR-31 A PIM File shall be UTF-8 JSON containing `"format": "pim/v1"`, the next Id, and every PIR with its Id, type, and fields. Each Alarm of an Event shall be stored with its kind.

FR-32 The system shall write the file atomically: it writes a temporary file in the same folder and then replaces the target with it. A failed write shall leave any earlier file intact and no temporary file behind. If the operating system refuses the write, the system shall report `cannot save <absolute path>: <reason>`, and the Dirty state shall not change.

FR-33 An empty path shall be rejected with `path is required`, and a path whose file name is empty or only `.pim` with `file name is required`.

Verification of FR-29 – FR-33 unit tests in `tests/unit/test_persist.py`, including `test_failed_replace_keeps_earlier_file_and_leaves_no_temp`; controller unit tests for operating-system errors; Terminal unit tests for the empty path.

6.7 Loading (US11)

FR-34 `load <path>` shall accept only a path that ends in `.pim`; any other path shall be rejected with `path must have a .pim extension` before the file is read. If the collection is dirty, the question of FR-37 shall come first. A successful load shall replace the Working Collection, the next Id, and the Bound File, and shall end any search. `<path>` takes the same forms as in FR-29. On a TTY, `load` without a path shall open the folder browser (FR-39); the line interface shall ask for a typed path. The status line shall name the absolute path that was read.

FR-35 Saving and then loading a collection shall give back the same PIRs with the same Ids, types, fields, alarm kinds, and Effective Alarm Times.

FR-36 A file that is missing, unreadable, not UTF-8, not valid JSON, not `pim/v1`, or that holds an invalid PIR shall be rejected with `not a PIM file: <reason>`, and the Working Collection shall stay unchanged. Invalid PIRs include:

- a missing required field;
- a duplicate or non-positive Id, or an Id that is not an integer;

- an alarm list that is not a list;
- an alarm out of range.

FR-37 If the collection is dirty, `load` and `quit` shall first ask the user to *save*, *discard*, or *cancel*, and *cancel* shall change nothing. On a TTY, end of input (Ctrl-D) and interrupt (Ctrl-C) shall follow the same rule as `quit`. When redirected input ends while the collection is dirty, no answer can follow, so the session shall end without saving and report `input ended: unsaved changes were not saved`.

Verification of FR-34 – FR-37 unit tests in `tests/unit/test_persist.py`; Terminal unit tests for the dirty prompts; e2e test `test_piped_input_ending_with_unsaved_changes_says_so` and the e2e load and quit scripts.

6.8 User interface

FR-38 The system shall accept the following commands:

- `create [type]`, `search [criterion]`, `clear`;
- a row number `<n>`, and `id <n>`;
- `modify`, `print`, `print all`, `delete`;
- `save`, `save as [path]`, `load [path]`;
- `dismiss`, `help`, and `quit`, with `q` and `exit` as aliases of `quit`.

An unknown command shall be reported as `unknown command: <text>` and change nothing. On a TTY, single-key accelerators (for example `c` create, `/` search, `w` save, `W` save as, `o` load) shall run the same commands, and `:` shall open a field for typing a full command.

Verification unit tests in `tests/unit/test_keys.py` and `test_unknown_command_and_non_print_clears_print_text`; e2e command scripts in `tests/e2e/test_commands.py`.

FR-39 **Folder browser**

On a TTY, a load or save-as path prompt shall show a folder browser. The browser shall list the parent folder (except at the root of the file system), subfolders, and `.pim` files, leaving out hidden entries. It shall start in the folder of the Bound File, or otherwise in the folder from which the system was started. Enter on a folder shall open it, and Enter on a `.pim` file shall use that file. In save mode, a "new file in this folder" entry shall ask for a file name. `/` or Tab shall switch to typing a path, and Esc shall cancel. The chosen path shall be handled exactly like a typed one (FR-29 – FR-37). A folder that cannot be read shall be reported inside the browser rather than as an error that ends the command. There shall be no graphical file dialog.

Verification unit tests for the browser in `tests/unit/test_file_browser.py`, the Terminal path prompts, and the curses browser keys.

FR-40 The screen shall show:

- a title line with the Bound File or "untitled" and a mark for unsaved changes; the full-screen interface also shows the HKT clock;
- the Alarm Alerts (FR-41);
- the Current Result as a table of Id, type, Display Name, and deadline or start;
- the details of the Selection;

- the status line.

A row number shall select by position in the Current Result and shall not be treated as an identity.

Verification unit tests in `tests/unit/test_layout.py`; integration tests `test_layout_shows_search_and_dirty_flag` and `test_select_row_is_not_identity`.

6.9 Alarm Alerts (derived from US4)

FR-41 While the program runs, it shall show an Alarm Alert for every alarm that is OVERDUE (Effective Alarm Time ≤ now) or SOON (Effective Alarm Time within the next 15 minutes), naming the Event and the time. The line interface shall list every alert; the full-screen interface shall show the first alert and the number of further alerts.

FR-42 The alerts shall update within 500 ms of an alarm becoming due, even when the user types nothing.

FR-43 The command `dismiss` shall hide the first alert. A dismissed alert shall stay hidden for the rest of the session, unless a load replaces the collection or the alarms of its Event are changed. Dismissals shall not be written to the PIM File, and the system shall use neither operating-system notifications nor a second process.

Verification of FR-41 – FR-43 unit tests for `due_alarms` with an injected time in `tests/unit/test_alarms.py`; e2e tests in `tests/e2e/test_alerts.py`; for FR-42, inspection of the 500 ms wait in both event loops.

6.10 Non-functional requirements

NFR-1 **Standard library**

The implementation shall use only the Python 3 standard library.

Verification inspection of every import.

NFR-2 **Platform**

The system shall run on macOS with Python 3.12 or newer, in Terminal or iTerm. The developer manual shall describe this platform only.

Verification `python3 -m unittest` and the demonstration script on macOS with Python 3.12; inspection of the developer manual.

NFR-3 **Structure**

All model code shall be in a package named `model`, which shall not import `view` or `controller`. The other components shall be in the packages `view` and `controller`.

Verification inspection of the imports.

NFR-4 **Reliability**

Invalid input or a failed command shall not change the Working Collection, shall not print a traceback, and shall not end the process. The system shall report exactly one English

message on the status line. This covers at least an unknown command, a missing required field, a bad Id or row number, a bad date or time, a search syntax error, a path without `.pim` on load, a missing file name, a missing Selection, and an attempt to change the type of a PIR. An unexpected internal error in a command shall be reported as `command failed`, and the session shall continue.

Verification unit, integration, and e2e error tests.

NFR-5 Data integrity

A failed save shall not damage the earlier file, and a failed load shall not change the Working Collection (FR-32, FR-36). The system shall never discard unsaved changes without asking or, for redirected input, without saying so (FR-37).

Verification unit tests `test_failed_replace_keeps_earlier_file_and_leaves_no_temp`, `test_bad_json_does_not_clobber_memory`, and `test_dirty_quit_save_discard_and_invalid_choice`; e2e test `test_piped_input_ending_with_unsaved_changes_says_so`.

NFR-6 Time

Every stored time shall carry a time zone. The default zone shall be HKT, and comparisons shall use instants (FR-11, FR-20). If the computer has no time-zone database, the system shall use a fixed UTC+8 offset for HKT instead of failing to start.

Verification unit tests `ParseDatetimeTests`, including `test_missing_zone_database_falls_back_to_fixed_utc_plus_8`.

NFR-7 Performance

With 10,000 PIRs in the Working Collection, each of the following shall finish within 1 second on the development Mac: a search, the alarm check, a save, and a load.

Verification measured on 24 Sep 2026 with 2,500 PIRs of each type, every Event having three alarms, taking the best of five runs. The search `type = event && description contains "comp" || alarm < 2026-10-01 00:00` took 2 ms, the alarm check 2 ms, a save 44 ms, and a load 24 ms.

NFR-8 Testability

Every model rule shall be testable through the public interface of `model`, without the user interface. The model unit tests shall use `unittest`, run automatically, pass, and execute every countable line of `model/`. Only `model/__init__.py`, which merely re-exports names, is excluded from the count.

Verification `python3 -m unittest`; `python3 tests/coverage_report.py`.

NFR-9 Usability

Prompts, messages, and manuals shall be in English. Each error message shall say what went wrong and shall name the field or value at fault when there is one, for example `description is required` or `no PIR with Id 9`.

Verification unit test `test_validation_messages_name_the_field_or_value_at_fault` and the error tests of NFR-4 assert the exact message text; inspection of the error table in the user manual.

NFR-10 Scope

The system shall not include any of the out-of-scope features listed in Section 2.2.

Verification inspection of the commands (FR-38) and of the source code against the list in Section 2.2.